WIKIPEDIA

# ext2

The **ext2** or **second extended file system** is a file system for the Linux kernel. It was initially designed by Rémy Card as a replacement for the extended file system (ext). Having been designed according to the same principles as the Berkeley Fast File System from BSD, it was the first commercial-grade filesystem for Linux.[4]

The canonical implementation of ext2 is the "ext2fs" filesystem driver in the Linux kernel. Other implementations (of varying quality and completeness) exist in GNU Hurd, MINIX 3, some BSD kernels, in MiNT, and as third-party Microsoft Windows and macOS drivers.

ext2 was the default filesystem in several Linux distributions, including Debian and Red Hat Linux, until supplanted more recently by ext3, which is almost completely compatible with ext2 and is a journaling file system. ext2 is still the filesystem of choice for flash-based storage media (such as SD cards and USB flash drives) because its lack of a journal increases performance and minimizes the number of writes, and flash devices have a limited number of write cycles. However, recent Linux kernels support a journal-less mode of ext4 which provides benefits not found with ext2.

| ext2 | |
|---|---|
| **Developer(s)** | Rémy Card |
| **Full name** | Second extended file system |
| **Introduced** | January 1993 with Linux |
| **Partition identifier** | `Apple_UNIX_SVR2` (Apple Partition Map)<br>`0x83` (Master Boot Record)<br>`EBD0A0A2-B9E5-4433-87C0-68B6B72699C7` (GPT) |
| **Structures** | |
| **Directory contents** | Table |
| **File allocation** | bitmap (free space), table (metadata) |
| **Bad blocks** | Table |
| **Limits** | |
| **Max. volume size** | 2–32 TiB |
| **Max. file size** | 16 GiB – 2 TiB |
| **Max. number of files** | $10^{18}$ |
| **Max. filename length** | 255 bytes |
| **Allowed characters in filenames** | All bytes except NUL ('\0') and '/' |
| **Features** | |
| **Dates recorded** | modification (mtime), attribute modification (ctime), access (atime) |
| **Date range** | December 14, 1901 - January 18, 2038 |

## Contents

**History**
**ext2 data structures**
- Inodes
- Directories
- Allocating data

**File-system limits**
**Compression extension**
**Under other operating systems**
**See also**
**References**
- Notes

**Further reading**
**External links**

## History

| | |
|---|---|
| **Date resolution** | 1 s |
| **File system permissions** | POSIX |
| **Transparent compression** | No (Available through patches) |
| **Transparent encryption** | No |
| **Other** | |
| **Supported operating systems** | Linux, BSD,[1][2] ReactOS,[3] Windows (through an IFS), macOS (through an IFS) |

The early development of the Linux kernel was made as a cross-development under the MINIX operating system. The MINIX file system was used as Linux's first file system. The Minix file system was mostly free of bugs, but used 16-bit offsets internally and thus had a maximum size limit of only 64 megabytes, and there was also a filename length limit of 14 characters[5]. Because of these limitations, work began on a replacement native file system for Linux.[5]

To ease the addition of new file systems and provide a generic file API, VFS, a virtual file system layer, was added to the Linux kernel. The extended file system (ext), was released in April 1992 as the first file system using the VFS API and was included in Linux version 0.96c. The ext file system solved the two major problems in the Minix file system (maximum partition size and filename length limitation to 14 characters), and allowed 2 gigabytes of data and filenames of up to 255 characters. But it still had problems: there was no support of separate timestamps for file access, inode modification, and data modification.

As a solution for these problems, two new filesystems were developed in January 1993 for Linux kernel 0.99: xiafs and the **second extended file system** (**ext2**), which was an overhaul of the extended file system incorporating many ideas from the Berkeley Fast File System. ext2 was also designed with extensibility in mind, with space left in many of its on-disk data structures for use by future versions.

Since then, ext2 has been a testbed for many of the new extensions to the VFS API. Features such as the withdrawn POSIX draft ACL proposal and the withdrawn extended attribute proposal were generally implemented first on ext2 because it was relatively simple to extend and its internals were well understood.

On Linux kernels prior to 2.6.17,[6] restrictions in the block driver mean that ext2 filesystems have a maximum file size of 2 TiB.

ext2 is still recommended over journaling file systems on bootable USB flash drives and other solid-state drives. ext2 performs fewer writes than ext3 because there is no journaling. As the major aging factor of a flash chip is the number of erase cycles, and as erase cycles happen frequently on writes, decreasing writes increases the life span of the solid-state device.[7] Another good practice for filesystems on flash devices is the use of the *noatime* mount option, for the same reason.

# ext2 data structures

The space in ext2 is split up into blocks. These blocks are grouped into block groups, analogous to cylinder groups in the Unix File System. There are typically thousands of blocks on a large file system. Data for any given file is typically contained within a single block group where possible. This is done to minimize the number of disk seeks when reading large amounts of contiguous data.

Each block group contains a copy of the superblock and block group descriptor table, and all block groups contain a block bitmap, an inode bitmap, an inode table, and finally the actual data blocks.

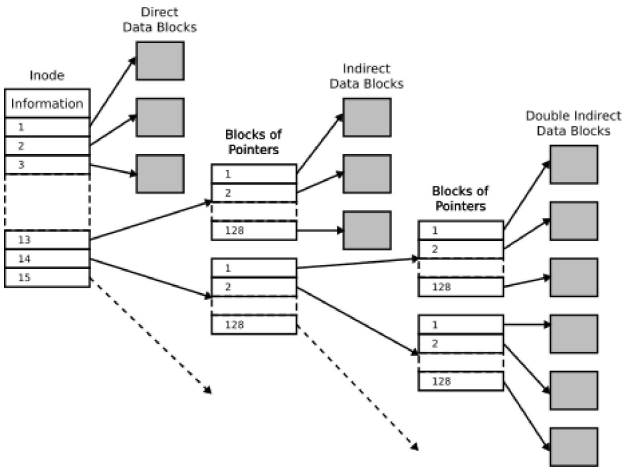
The superblock contains important information that is crucial to the booting of the operating system. Thus backup copies are made in multiple block groups in the file system. However, typically only the first copy of it, which is found at the first block of the file system, is used in the booting.

The group descriptor stores the location of the block bitmap, inode bitmap, and the start of the inode table for every block group. These, in turn, are stored in a group descriptor table.

## Inodes

Every file or directory is represented by an inode. The term "inode" comes from "index node" (over time, it became i-node and then inode).[8] The inode includes data about the size, permission, ownership, and location on disk of the file or directory.

Example of ext2 inode structure:




Quote from the Linux kernel documentation for ext2:

> **"There are pointers to the first 12 blocks which contain the file's data in the inode. There is a pointer to an indirect block (which contains pointers to the next set of blocks), a pointer to a doubly indirect block and a pointer to a trebly indirect block."**[9]

Thus, there is a structure in ext2 that has 15 pointers. Pointers 1 to 12 point to direct blocks, pointer 13 points to an indirect block, pointer 14 points to a doubly indirect block, and pointer 15 points to a triply indirect block.

## Directories

Each directory is a list of directory entries. Each directory entry associates one file name with one inode number, and consists of the inode number, the length of the file name, and the actual text of the file name. To find a file, the directory is searched front-to-back for the associated filename. For reasonable directory sizes, this is fine. But for very large directories this is inefficient, and ext3 offers a second way of storing directories (HTree) that is more efficient than just a list of filenames.

The root directory is always stored in inode number two, so that the file system code can find it at mount time. Subdirectories are implemented by storing the name of the subdirectory in the name field, and the inode number of the subdirectory in the inode field. Hard links are implemented by storing the same inode number with more than one file name. Accessing the file by either name results in the same inode number, and therefore the same data.

The special directories "." (current directory) and ".." (parent directory) are implemented by storing the names "." and ".." in the directory, and the inode number of the current and parent directories in the inode field. The only special treatment these two entries receive is that they are automatically

created when any new directory is made, and they cannot be deleted.

### Allocating data

When a new file or directory is created, ext2 must decide where to store the data. If the disk is mostly empty, then data can be stored almost anywhere. However, clustering the data with related data will minimize seek times and maximize performance.

ext2 attempts to allocate each new directory in the group containing its parent directory, on the theory that accesses to parent and children directories are likely to be closely related. ext2 also attempts to place files in the same group as their directory entries, because directory accesses often lead to file accesses. However, if the group is full, then the new file or new directory is placed in some other non-full group.

The data blocks needed to store directories and files can be found by looking in the data allocation bitmap. Any needed space in the inode table can be found by looking in the inode allocation bitmap.

## File-system limits

The reason for some limits of ext2 are the file format of the data and the operating system's kernel. Mostly these factors will be determined once when the file system is built. They depend on the block size and the ratio of the number of blocks and inodes. In Linux the block size is limited by the architecture page size.

Theoretical ext2 limits under Linux[10]

| **Block size:** | 1 KiB | 2 KiB | 4 KiB | 8 KiB |
|---|---|---|---|---|
| **max. file size:** | 16 GiB | 256 GiB | 2 TiB | 2 TiB |
| **max. filesystem size:** | 4 TiB | 8 TiB | 16 TiB | 32 TiB |

There are also some userspace programs that cannot handle files larger than 2 GiB.

If $b$ is the block size, the maximal file size is limited to min( $((b/4)^3 + (b/4)^2 + b/4 + 12) \times b$, $(2^{32} - 1) \times 512$ ) due to the i_block structure (an array of direct/indirect EXT2_N_BLOCKS) and i_blocks (32-bit integer value) representing the number of 512-byte "blocks" in the file.

The maximal number of sublevel-directories is 31998, due to the link-count limit. Directory indexing is not available in ext2, so there are performance issues for directories with a large number of files (>10,000). The theoretical limit on the number of files in a directory is $1.3 \times 10^{20}$, although this is not relevant for practical situations.

Note: In Linux 2.4 and earlier, block devices were limited to 2 TiB, limiting the maximal size of a partition, regardless of block size.

## Compression extension

**e2compr** is a modification to the ext2 driver in the Linux kernel to support compression and decompression of files by the file system, without any support by user applications. e2compr is a small patch against ext2.

e2compr compresses only regular files; the administrative data (superblock, inodes, directory files, etc.) are not compressed (mainly for safety reasons). Access to compressed blocks is provided for read and write operations. The compression algorithm and cluster size is specified on a per-file basis. Directories can also be marked for compression, in which case every newly created file in the directory will be automatically compressed with the same cluster size and the same algorithm that was specified for the directory.

e2compr is not a new file system. It is only a patch to ext2 made to support the EXT2_COMPR_FL flag. It does not require user to make a new partition, and will continue to read or write existing ext2 file systems. One can consider it as simply a way for the read and write routines to access files that could have been created by a simple utility similar to gzip or compress. Compressed and uncompressed files coexist nicely on ext2 partitions.

The latest e2compr-branch is available for current releases of Linux 2.4, 2.6, and 3.0. The latest patch for Linux 3.0 was released in August 2011 and provides multicore and High memory support. There are also branches for Linux 2.0 and 2.2.

# Under other operating systems

Access to ext2 partitions under Microsoft Windows is possible through an Installable File System, such as ext2ifs[11] or ext2Fsd.[12] Filesystem in Userspace can be used on macOS.[13]

# See also

- e2fsprogs
- StegFS – a steganographic file system based on ext2
- cloop
- List of file systems
- Comparison of file systems

# References


1. "Chapter 8. Disks, File Systems, and Boot Loaders" (https://www.freebsd.org/doc/en_US.ISO8859-1/books/faq/disks.html#idp59236072). *www.freebsd.org*. Retrieved 11 April 2018.
2. "ext2fs(5)" (https://www.freebsd.org/cgi/man.cgi?query=ext2fs&sektion=5&manpath=freebsd-release-ports). *www.freebsd.org*. Retrieved 11 April 2018.
3. "ReactOS 0.4.2 Released" (https://reactos.org/project-news/reactos-042-released). *reactos.org*. Retrieved 17 August 2016.
4. Mauerer, Wolfgang (2010). *Professional Linux Kernel Architecture* (https://books.google.com/books?id=-6zvRFEfQ24C&pg=PT490). John Wiley & Sons.
5. Strobel, Stefan; Uhl, Thomas (1994). *Linux—Unleashing the Workstation in Your PC*. Springer-Verlag. p. 54.
6. linux/kernel/git/torvalds/linux-2.6.git/commitdiff: (https://archive.is/20120709233348/http://git.kernel.org/?p=linux/kernel/git/torvalds/linux-2.6.git;a=commitdiff;h=a0f62ac6362c168754cccb36f196b3dfbddc3bc3), [PATCH] 2TB files: add blkcnt_t, Author:Takashi Sato, 26 Mar 2006 09:37:52 +0000 (01:37 -0800) — Commit allowing for large files, git.kernel.org
7. "Solid State Disk Drives" (http://www.supertalent.com/tools/ssd.php). Super Talent Technology.
8. "Programmer's Journal", Volume 5, 1987, p. 174
9. "The Second Extended File System" (https://www.nongnu.org/ext2-doc/ext2.html). *www.nongnu.org*. Retrieved 2019-12-11.
10. "File system guide" (https://web.archive.org/web/20110917065413/http://linuxreviews.org/sysadmin/filesystems/). Linux Reviews. Archived from the original (http://linuxreviews.org/sysadmin/filesystems/) on 2011-09-17. Retrieved 2008-01-24.
11. Randall, Neil (September 18, 2007). "Linux: You Can Do It!" (https://books.google.com/books?id=ecwM7EB9SPMC&pg=PA76). *PC Magazine*. Vol. 26 no. 18. p. 76.
12. Wallen, Jack (September 17, 2012). "Enable the mounting of ext2/3 file systems on a Windows machine" (https://www.techrepublic.com/blog/tr-dojo/enable-the-mounting-of-ext2-3-file-systems-on-a-windows-machine/). *TechRepublic*. Retrieved January 17, 2016.

13. Kessler, Topher (June 22, 2012). "How to manage Ext2/Ext3 disks in OS X" (https://www.cnet.com/how-to/how-to-manage-ext2ext3-disks-in-os-x/). *CNET*. Retrieved January 17, 2016.

### Notes

- Card, Rémy. & Ts'o, Theodore. & Tweedie, Stephen (1994). "Design and implementation of the second extended filesystem" (http://web.mit.edu/tytso/www/linux/ext2intro.html). *Proceedings of the First Dutch International Symposium on Linux*. ISBN 90-367-0385-9.
- Sourceforge e2compr project (http://sourceforge.net/projects/e2compr)
- Sourceforge e2compr documentation (http://e2compr.sourceforge.net/)
- Sourceforge e3compr project page, ext3 compression, alpha (http://sourceforge.net/projects/e3compr)
- Dr. Dobb's Data Compression Newsletter Issue #46 - September 2003 (http://www.ddj.com/showArticle.jhtml?articleID=184405431&queryText=seen)

## Further reading

- John Newbigin. "John's spec of the second extended filesystem" (http://uranus.chrysocome.net/explore2fs/es2fs.htm).
- Dave Poirier (2009). "The Second Extended File System: Internal Layout" (http://www.nongnu.org/ext2-doc/index.html).
- Theodore Ts'o; Stephen Tweedie (June 2002). "Planned Extensions to the Linux Ext2/Ext3 Filesystem" (http://e2fsprogs.sourceforge.net/extensions-ext23/). *USENIX 2002 Annual Technical Conference*.
- Larry Ayers (June 1997). "E2compr: Transparent File Compression for Linux" (https://web.archive.org/web/20091117121024/http://linuxgazette.net/issue18/e2compr.html). *Linux Gazette* (18). Archived from the original (http://linuxgazette.net/issue18/e2compr.html) on 2009-11-17.
- Charles Cong; Jeremy H. Brown (1997-12-26). "A System for Transparent File Compression With Caching Under Linux". CiteSeerX 10.1.1.51.4780 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.51.4780).
- Charles Cong; Jeremy H. Brown (1997-10-22). "A Survey of Modern File Compression Techniques". CiteSeerX 10.1.1.50.9847 (https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.50.9847).

## External links

- ext2fs user-space tools (http://e2fsprogs.sourceforge.net/ext2.html)
- Ext2Fsd (https://github.com/matt-wu/Ext3Fsd) GPL ext2/ext3 file system driver for Windows 2000/XP/2003/VISTA/2008 (opensource, supports read & write, works with FreeOTFE)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Ext2&oldid=938387663"

**This page was last edited on 30 January 2020, at 21:43 (UTC).**

Text is available under the Creative Commons Attribution-ShareAlike License; additional terms may apply. By using this site, you agree to the Terms of Use and Privacy Policy. Wikipedia® is a registered trademark of the Wikimedia Foundation, Inc., a non-profit organization.